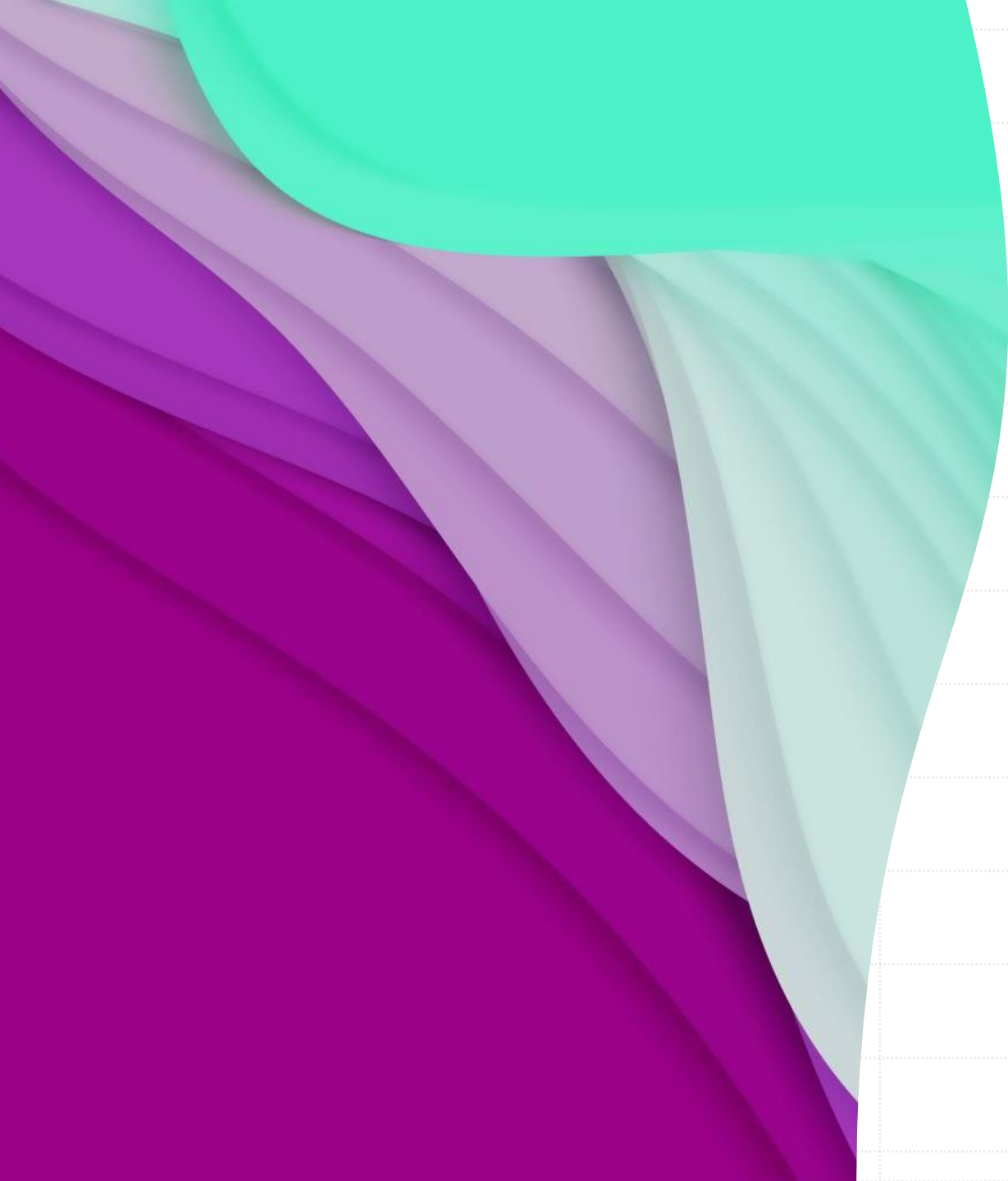




Python Programming Bootcamp – Week 3, Day 4

Error Handling and Debugging in Python

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering,
Science and Technology



Learning Objectives

- Understand Python's error types and common debugging strategies.
- Learn how to handle exceptions using try, except, finally blocks.
- Apply error handling in Python programs.
- Explore debugging techniques to identify and fix errors.



Types of Errors in Python

- **Syntax Errors:** Occur when the code violates Python's syntax rules.

```
print("Hello)
```

- **Runtime Errors:** Errors that occur while the program is running, like division by zero.

```
print(10 / 0)
```

- **Logical Errors:** Errors that cause the program to behave incorrectly, without crashing.



Introduction to Exception Handling

- **Definition:** Exceptions are events that can alter the normal flow of a program.
- **Why Handle Exceptions?:** To prevent the program from crashing and to provide informative feedback to users.



try, except, and finally Blocks

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```



Handling Multiple Exceptions

```
try:  
    num = int(input("Enter a number: "))  
except (ValueError, TypeError):  
    print("Invalid input!")
```



The else Clause in Exception Handling

- **Definition:** Executes if no exceptions are raised in the **try** block.

```
try:  
    result = 10 / 2  
except ZeroDivisionError:  
    print("Cannot divide by zero!")  
else:  
    print(f"Result: {result}")
```



Raising Exceptions

- You can raise your own exceptions using the `raise` keyword.

```
num = int(input('Please Enter an Even Number: '))  
if num%2 !=0:  
    raise ValueError('The Number is ODD')
```


Hands-On Exercise 1

- **Task:** Write a Python program that takes input from the user and raises an exception if the input is not a number.

"Caught an exception"





The finally Clause

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Invalid number!")
finally:
    print("This will always execute.")
```



Debugging Techniques

- **Print Debugging:** Use `print()` statements to understand the flow of the program.
- **Using Breakpoints:** Set breakpoints in your IDE to pause execution and inspect variables.
- **Logging:** Use Python's logging module to keep track of events and errors in a file.



Hands-On Exercise 2

- **Task:** Write a Python program that divides two numbers. Handle division by zero and other invalid inputs using `try-except`.



Debugging with assert

- Definition: assert statements check if a condition is true, raising an AssertionError if it is false.

```
assert 2 + 2 == 3, "Math error!"
```



Using the `pdb` Module for Debugging

- **Definition:** `pdb` is Python's built-in debugger.

```
import pdb
pdb.set_trace() # This will pause the program execution for debugging
```



Hands-On Group Activity

- **Task:** Create a Python program with intentional errors. Debug it using the techniques learned today.

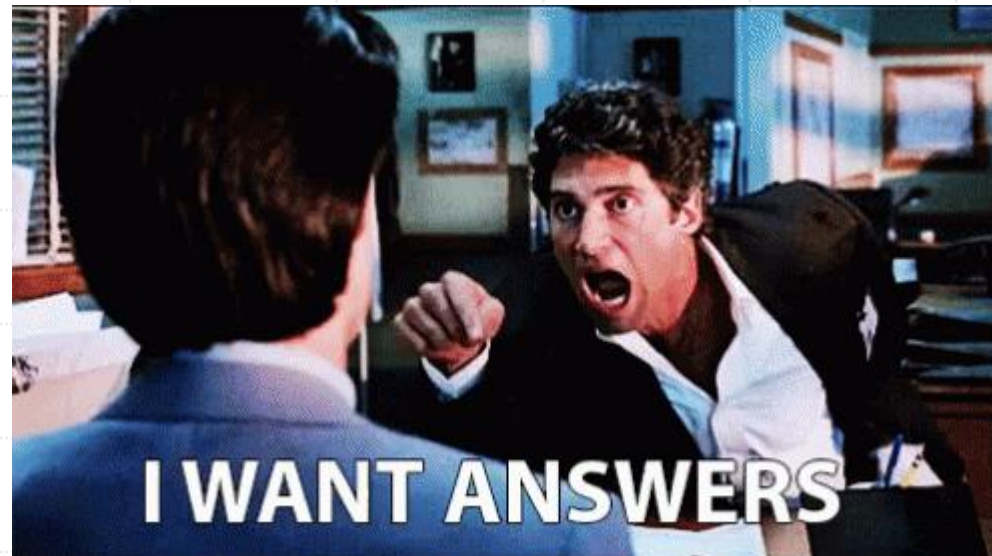


Recap and Key Takeaways

- **Key Points:**

- Exceptions help handle runtime errors without crashing the program.
- The try-except block is essential for error handling.
- Debugging involves identifying errors and finding solutions efficiently.

Q&A





MATCH THE SOLUTIONS 🙄



EXERCISE 😊

Step 1: Function to take input and check if it is a number

```
def check_number():
```

```
    user_input = input("Enter a number: ")
```

Step 2: Check if the input is a number

```
try:
```

```
    number = int(user_input)
```

```
    print(f"You entered the number: {number}")
```

```
except ValueError:
```

```
    # Raise an exception if the input is not a number
```

```
    print(f"'{user_input}' is not a valid number!")
```

Step 3: Call the function to test it

```
check_number()
```

EXERCISE



Step 1: Function to perform division with error handling

```
def divide_numbers():
    try:
        # Step 2: Take user input for two numbers
        num1 = float(input("Enter the first number: "))
        num2 = float(input("Enter the second number: "))
        # Step 3: Perform the division
        result = num1 / num2
        # Step 4: Print the result if no exceptions occur
        print(f"The result of {num1} divided by {num2} is {result}")
    except ZeroDivisionError:
        # Handle division by zero
        print("Error: Division by zero is not allowed!")
    except ValueError:
        # Handle invalid input (non-numeric values)
        print("Error: Please enter valid numeric values!")
    finally:
        # This will always execute
        print("Operation completed.")

# Step 5: Call the function to test it
divide_numbers()
```

EXERCISE



```
# Step 1: A buggy program for debugging
def buggy_function():
    try:
        # Step 2: Take two numbers from the user
        num1 = int(input("Enter the first number: ")) # Potential ValueError
        num2 = int(input("Enter the second number: ")) # Potential ValueError
        # Step 3: A mistake in division (missing error handling)
        result = num1 / num2
        print(f"Result: {result}")
        # Step 4: Intentional logical error in multiplication
        wrong_multiplication = num1 * 'abc' # Should raise a TypeError
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed!")
    except ValueError:
        print("Error: Invalid input. Please enter a valid number.")
    except TypeError:
        print("Error: Type mismatch in multiplication!")
    finally:
        print("End of program.")
# Step 5: Call the buggy function
buggy_function()
```